

提高级 CSP-S 第 4 套初赛模拟试题

一、单项选择题(共 15 题,每题 2 分,共计 30 分;每题有且仅有一个正确选项)

- 在以下各项中,()不是 CPU 的组成部分。
A. 控制器 B. 运算器 C. 寄存器 D. 主板
- $(2017)_8 + (1234)_{10}$ 的结果是()。
A. $(8E2)_{16}$ B. $(100011100001)_2$
C. $(8E0)_{16}$ D. $(100011100011)_2$
- 设 $A=B=True, C=D=False$, 逻辑运算表达式值为假的是()。
A. $(\neg A \wedge B) \vee (C \wedge D \vee A)$ B. $\neg (((A \wedge B) \vee C) \wedge D)$
C. $A \wedge (B \vee C \vee D) \vee D$ D. $(A \wedge (D \vee C)) \wedge B$
- 若已知一个栈的入栈序列是 $1, 2, 3, \dots, n$, 其输出序列为 $p_1, p_2, p_3, \dots, p_n$, 若 $p_1 = n$, 则 p_i 为()。
A. i B. $n-i$ C. $n-i+1$ D. 不确定
- 设 $S = \text{"abbccce"}$, 不同的非空子串个数有()个。
A. 19 B. 17 C. 16 D. 18
- C++ 中, $125 \sim 2$ 的值是()。
A. 127 B. 128 C. 15625 D. 126
- 设某算法的计算时间表示为递推关系式 $T(n) = 2T(n/2) + n$ (n 为正整数) 及 $T(1) = 1$, 则该算法的时间复杂度为()。
A. $O(n)$ B. $O(n^2)$ C. $O(n \log n)$ D. $O(\log n)$
- 下列有关二叉树的叙述, 不正确的是()。
A. 二叉树的深度为 k , 那么最多有 $2^k - 1$ 个节点 ($k \geq 1$)
B. 在二叉树的第 i 层上, 最多有 2^{i-1} 个节点 ($i \geq 1$)
C. 完全二叉树一定是满二叉树
D. 堆是完全二叉树
- 下列排序算法中, 平均时间复杂度是 $O(n^2)$ 的是()。
A. 快速排序 B. 插入排序 C. 归并排序 D. 堆排序
- 下列哪些图一定可以进行黑白染色, 使得相邻节点的颜色不同()。
A. 树 B. 基环树 C. 连通图 D. 欧拉图
- 下列不是操作系统的有()。
A. Linux B. Windows C. Android D. WPS
- 下列关于算法的叙述中, 错误的是()。
A. 算法代表着用系统的方法描绘解决问题的策略机制。
B. 一个算法的好坏, 只需要从时间复杂度这一方面进行考虑
C. 一个算法必须具有有穷性、可行性、正确性、输入和输出
D. 对于一些 np 完全问题, 现在未能找到有效的算法解决
- 下列叙述中正确的是()。
A. 线性表是线性结构 B. 栈与队列是非线性结构
C. 线性链表是非线性结构 D. 二叉树是线性结构
- 链表的()操作需要 $O(n)$ 的时间复杂度实现。
A. 插入 B. 定位 C. 删除 D. 合并
- NOI 比赛, 下列选项中可以带入考场的是()。
A. 键盘 B. 参考书籍 C. U 盘 D. 铅笔

二、阅读程序(程序输入不超过数组或字符串定义的范围;判断题正确的填√,错误的填×;除特殊说明外,判断题每题 1.5 分,选择题每题 4 分,共计 40 分)

1.

```
01  #include<iostream>
02  using namespace std;
03  int n,k,ans;
04  int main()
05  {
06      cin>>n>>k;
07      int ans=0;
08      while(n)
09      {
10          if(n%k>0) ans++;
11          n/=k;
12      }
13      cout<<ans<<endl;
14      return 0;
15  }
```

●判断题

- (1)(1 分)把第 10 行 $n\%k>0$ 改成 $n\%k$ 不影响程序运行结果。()
- (2)输入的 k 需要大于 1。()
- (3)输入的 n 需要大于 0。()
- (4)该算法的时间复杂度为 $O(\log_k n)$ 。()

●选择题

- (5)输入 125 5 的输出结果为()。
- A. 1 B. 2 C. 3 D. 4
- (6)当 n 在 `int` 范围内时,输出的最大值为()。
- A. 29 B. 30 C. 31 D. 32

2.

```
01  #include<iostream>
02  using namespace std;
03  int equationCount(int n,int m)
04  {
05      if(n==1 || m==1)
06          return 1;
07      else if(n<m)
08          return equationCount(n,n);
09      else if(n==m)
10          return 1+equationCount(n,n-1);
11      else
12          return equationCount(n,m-1)+equationCount(n-m,m);
13  }
14  int main()
15  {
16      int n;
17      cin>>n;
```

```

18     cout<<equationCount(n,n)<<endl;
19     return 0;
20 }

```

●判断题

- (1)(1分)输入的 n 必须为正整数。()
- (2)把第 09 行的“else if($n==m$)”和第 10 行的“return 1+equationCount($n,n-1$);”去掉,不影响程序运行结果。()
- (3)把第 18 行的“ n,n ”改成“ $n,n+1$ ”不影响程序运行结果。()
- (4)把第 07 行的“else if($n<m$)”和第 08 行的“return equationCount(n,n);”去掉,不影响程序运行结果。()

●选择题

- (5)输入 7 的输出结果为()。
- A. 12 B. 13 C. 14 D. 15
- (6)该算法的时间复杂度为()。
- A. $O(\log n)$ B. $O(n)$ C. $O(n^2)$ D. 以上都不是

3.

```

01  #include<iostream>
02  using namespace std;
03  const int maxn=100000;
04  int a[maxn],b[maxn],n;
05  int Search(int num,int low,int high)
06  {
07      int mid;
08      while(low<=high)
09      {
10          mid=(low+high)/2;
11          if(num>=b[mid]) low=mid+1;
12          else high=mid-1;
13      }
14      return low;
15  }
16  int main()
17  {
18      int len,pos;
19      cin>>n;
20      for(int i=1;i<=n;i++)
21          cin>>a[i];
22      b[1]=a[1];
23      len=1;
24      for(int i=2;i<=n;i++)
25      {
26          if(a[i]>=b[len])
27          {
28              len++;
29              b[len]=a[i];
30          }

```

```

31         else
32         {
33             pos=Search(a[i],1,len);
34             b[pos]=a[i];
35         }
36     }
37     cout<<len<<endl;
38     return 0;
39 }

```

● 判断题

- (1) (1分) 输入的 $a[i]$ 必须在 $[1, n]$ 范围内。()
- (2) (1分) 把第 11 行的“mid+1”改成“mid”不影响程序运行结果。()
- (3) 当数组 a 单调不降时输出为 1。()
- (4) 数组 b 内的元素始终单调不降。()

● 选择题

- (5) 当输入第一行为 20, 第二行为 1 20 2 19 3 18 4 17 ... 9 12 10 11 时, 输出为()。
- A. 1 B. 10 C. 11 D. 20
- (6) 该算法的时间复杂度为()。
- A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(n^2 \log n)$

三、完善程序(单选题, 每题 3 分, 共计 30 分)

1. 给一个矩阵 $N * M$, 给你第 i 行 1 的个数和位置, 让你选一些行精确覆盖 M 列(精确覆盖: 每列有且只有 1 个 1)。

例如: 如下的矩阵:

```

11100001
10001110
10010110
00010010
00001100

```

就包含了这样一个集合(第 1、4、5 行)。

Input

多组数据, 对于每组数据:

第一行两个整数 N, M ;

接下来 N 行, 每行开头一个整数 x , 表示该行上 1 的个数, 接下来 x 个整数, 每个 1 的位置。

Output

如果有解随意输出一组集合, 否则输出 NO, 中间空格隔开。

```

01 #include<bits/stdc++.h>
02 using namespace std;
03 #define ll long long
04 #define inf 1000000000
05 #define N 2005
06 #define M 2000005
07 int read()
08 {
09     int x=0, f=1; char ch=getchar();
10     while(ch<'0' || ch>'9') {if(ch=='-') f=-1; ch=getchar();}
11     while(ch>='0' && ch<='9') {x=x*10+ch-'0'; ch=getchar();}

```

```

12     return x*f;
13 }
14 int n,m;
15 int h[N],s[N],q[N];
16 int u[M],d[M],L[M],R[M],C[M],X[M];
17 void del(int c) //delete ROW C
18 {
19     ①
20     ②
21     for(int i=d[c];i!=c;i=d[i])
22         for(int j=R[i];j!=i;j=R[j])
23             u[d[j]]=u[j],d[u[j]]=d[j],s[C[j]]--;
24 }
25 void add(int c)
26 {
27     L[R[c]]=R[L[c]]=c;
28     for(int i=u[c];i!=c;i=u[i])
29         for(int j=L[i];j!=i;j=L[j])
30             u[d[j]]=d[u[j]]=j,s[C[j]]++;
31 }
32 void link(int r,int c)
33 {
34     static int size=0;size++;
35     X[size]=r;C[size]=c;
36     s[c]++;
37     d[size]=d[c];
38     u[size]=c;u[d[size]]=size;
39     d[u[size]]=size;
40     if(h[r]==-1)
41         h[r]=L[size]=R[size]=size;
42     else
43     {
44         R[size]=R[h[r]];
45         L[size]=h[r];
46         L[R[size]]=size;
47         R[L[size]]=size;
48     }
49 }
50 bool dance(int k)
51 {
52     if(R[0]==0)
53     {
54         printf("%d",k);
55         for(int i=1;i<=k;i++)
56             printf("%d",X[q[i]]);
57         puts("");

```

```

58         return 1;
59     }
60     int mn=inf,c;
61     for(int i=R[0];i;i=R[i])
62         if(s[i]<mn)mn=s[i],c=i;
63     ③
64     for(int i=d[c];i!=c;i=d[i])
65     {
66         q[k+1]=i;
67         for(int j=R[i];j!=i;j=R[j]) ④
68             if(dance(k+1)) return 1;
69         for(int j=L[i];j!=i;j=L[j]) ⑤
70     }
71     add(c);
72     Return 0;
73 }
74 int main()
75 {
76     while (scanf ("%d%d",&n,&m) !=EOF)
77     {
78         for(int i=0;i<=m;i++)
79         {
80             d[i]=u[i]=i;
81             L[i+1]=i;R[i]=i+1;
82             s[i]=0;
83         }
84         R[m]=0;size=m;
85         int x,y;
86         for(int i=1;i<=n;i++)
87         {
88             h[i]=-1;
89             x=read();
90             while(x--)
91             {
92                 y=read();
93                 link(i,y);
94             }
95         }
96         if(!dance(0))puts("NO");
97     }
98     return 0;
99 }

```

(1)①处应填()。

A. L[L[c]]=R[c];

B. L[R[c]]=L[c];

C. R[L[c]]=R[c];

D. R[R[c]]=L[c];

(2) ②处应填()。

A. $L[L[c]] = R[c]$;

B. $L[R[c]] = L[c]$;

C. $R[L[c]] = R[c]$;

D. $R[R[c]] = L[c]$;

(3) ③处应填()。

A. $\text{del}(c)$;

B. $\text{add}(c)$;

C. $\text{del}(mn)$;

D. $\text{add}(mn)$;

(4) ④处应填()。

A. $\text{add}(j)$;

B. $\text{del}(j)$;

C. $\text{add}(C[j])$;

D. $\text{del}(C[j])$;

(5) ⑤处应填()。

A. $\text{add}(j)$;

B. $\text{del}(j)$;

C. $\text{add}(C[j])$;

D. $\text{del}(C[j])$;

2. (树的直径) 给定一棵 n 个节点的树, 每条边有个长度 w_i , 求这棵树直径的长度。树的直径是指树的最长简单路。

做法: 两次 BFS。开始任选一点 u 作为起点进行 BFS, 找到最远的一点 s , 再从 s 再次 BFS 找到最远的一点 t 。 $s-t$ 两点的路径长度就是直径的长度。

```
01 #include<iostream>
02 #include<cstring>
03 using namespace std;
04 const int inf=0x3f3f3f3f; //假设的无穷大值,具体数值为 1061109567
05 const int maxn=1005;
06 struct Node{
07     int to,w,next;
//临接表节点,to 表示这条边的终点,w 表示这条边的长度,next 表示下一条边的编号
08 }edge[maxn*2];
09 int head[maxn],tot; //head[u]表示 u 的临接表头节点的标号
10 int n; //节点树
11 int dis[maxn]; //离起点的距离
12 bool vis[maxn]; //某个点是否已经拜访过
13 int que[maxn],first,last; //队列
14 void init()
15 {
16     memset(head,-1,sizeof(head));
17     tot=0;
18 }
19 void addedge(int u,int v,int w)
20 {
21     edge[tot].to=v;
22     edge[tot].w=w;
23     edge[tot].next=head[u];
24     head[u]=tot++;
25 }
26 int BFS(int u)
27 {
28     first=last=0;
29     memset(dis,inf,sizeof(dis));
30     memset(vis,0,sizeof(vis));
31     dis[u]=0;
32     vis[u]=1;
```

```

33     que[ last++]=u;
34     while( ① )
35     {
36         u=que[ first++];
37         for(int i=head[u];i!=-1;i=edge[i].next)
38         {
39             int v=edge[i].to;
40             if( ② )
41             {
42                 vis[v]=1;
43                 que[ last++]=v;
44                 ③ ;
45             }
46         }
47     }
48     int tmp=1;
49     for(int i=2;i<=n;i++)
50         if( ④ ) tmp=i;
51     return tmp;
52 }
53 int main()
54 {
55     int u,v,w,s,t;
56     cin>>n;
57     init();
58     for(int i=1;i<n;i++)
59     {
60         cin>>u>>v>>w;
61         addedge(u,v,w);
62         addedge(v,u,w);
63     }
64     s=BFS(1);
65     ⑤ ;
66     cout<<dis[t]<<endl;
67     return 0;
68 }

```

(1) ①处应填()。

- A. first<last B. first<=last C. first<last-1 D. last==n

(2) ②处应填()。

- A. !vis[v] B. vis[v] C. vis[u] D. dis[v]

(3) ③处应填()。

- A. dis[v]=dis[u]+1 B. dis[v]=dis[u]+edge[i].w
C. dis[u]=dis[v]+1 D. dis[u]=dis[v]+edge[i].w

(4) ④处应填()。

- A. dis[i]<dis[tmp] B. dis[i]<tmp
C. dis[i]>dis[tmp] D. dis[i]==tmp

(5) ⑤处应填()。

- A. t=BFS(s) B. t=BFS(1) C. t=BFS(t) D. s=BFS(t)